
HYBRID APPROACHES CONSISTENTLY BEAT OFFLINE RLHF METHODS

Anthony Kuang, Richik Pal & Eddie Cui

Department of Computer Science

University of California, Berkeley

Berkeley, CA 94709, USA

{anthonykuang, richik.pal, eddie.cui}@berkeley.edu

ABSTRACT

We study RLHF for open-ended instruction following on a 5k-example WildChat benchmark with Qwen2.5-1.5B-Instruct as the base model, implementing six contemporary baselines (DPO, IPO, AOT, GRPO, DrGRPO, GSPO) and a Bradley-Terry reward model. Building on the observation that DPO, IPO, and AOT optimize complementary notions of improvement, our Part 2 investigation introduces three algorithmic changes. First, a confidence-aware pair-weighting scheme that scales the DPO loss per preference pair by an estimate of label confidence, softening reliance on noisy WildChat judgments. Second, a hybrid AOT-DPO loss with a cosine schedule on the mixing coefficient α that keeps α near zero during the early noise-robust AOT-dominant phase and ramps it sharply near the end. Third, reward-model reranking over a pool of four diverse policy checkpoints. Across three random seeds the combined pipeline reaches a GPT-5.4 head-to-head win rate of 0.865 ± 0.015 against the frozen base model, an absolute improvement of 0.102 over the strongest Part 1 baseline mean, with 60.2% of the reranker’s winning candidates traced to the novel objectives.

1 INTRODUCTION

Reinforcement Learning from Human Feedback (RLHF) has emerged as the dominant paradigm for aligning pretrained language models with human preferences (Ouyang et al., 2022). Existing approaches fall into two broad categories: *offline* methods, which fine-tune directly on fixed preference datasets, and *online* methods, which iteratively sample from the current policy and update it using reward-model feedback. Within the offline family, DPO (Rafailov et al., 2023), IPO (Azar et al., 2024), and AOT (Melnik et al., 2024) share the same preference-pair data but differ in loss structure and comparison granularity. Within the online family, GRPO (Shao et al., 2024), DrGRPO (Liu et al., 2025), and GSPO (Zheng et al., 2025) all employ a PPO-style clipped surrogate but diverge in how they normalize advantages and at what granularity they apply clipping.

In this work, we implement and evaluate all 6 policy methods together with a reward model on a curated 5k-example open-ended instruction-following benchmark curated from WildChat, using Qwen2.5-1.5B-Instruct as the base model. We make the following contributions:

1. We implement and evaluate 6 contemporary RLHF algorithms together with a Bradley-Terry reward model, under a single fixed benchmark. We identify DPO as the strongest offline baselines and GRPO as the strongest online baseline, and trace these results to specific properties of the respective objectives.
2. We investigate a hybrid offline algorithm, convincingly beating offline baselines. Then, we discuss which interventions succeeded, which failed, and why.

2 OFFLINE METHODS

Background. The three offline objectives in this section share the same data and reference policy. The training set $\mathcal{D}$ is a collection of triples (x, y^+, y^-) , where x is a prompt and $y^+ \succ y^-$ is a

human-judged preference. A frozen reference policy π_{ref} supplies a fixed comparison point. Each objective derives a scalar signal from the policy-versus-reference log-likelihood gap and uses it to push mass toward chosen completions and away from rejected ones. The methods differ in the loss geometry applied to that signal and in the granularity at which signals are compared.

DPO. DPO (Rafailov et al., 2023) reformulates the KL-constrained reward maximization objective as a direct policy-optimization problem. By inverting the closed-form optimal policy of the KL-regularized objective and substituting into the Bradley-Terry preference model, the intractable partition function cancels and the preference probability is expressed entirely in terms of policies. Defining the reference-corrected preference margin $\Delta_\theta(x, y^+, y^-) = (\log \pi_\theta(y^+ | x) - \log \pi_\theta(y^- | x)) - (\log \pi_{\text{ref}}(y^+ | x) - \log \pi_{\text{ref}}(y^- | x))$, the DPO loss is

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x, y^+, y^-) \sim \mathcal{D}} [\log \sigma(\beta \Delta_\theta)].$$

The gradient increases the log-likelihood of chosen responses and decreases that of rejected ones, with each example weighted by how incorrectly the implicit reward currently ranks the pair.

IPO. DPO’s logistic on $\beta \Delta_\theta$ is minimized as $\Delta_\theta \rightarrow \infty$, so on near-deterministic preferences the policy can collapse the rejected likelihood to zero without effective KL pushback. Azar et al. (2024) formalize this through the Ψ PO framework and replace the logistic with a squared regression to a finite target. Under the KL-constrained objective the optimal policy satisfies $\Delta_{\theta^*} = \frac{1}{2\beta}$ for every pair, which IPO optimizes directly:

$$\mathcal{L}_{\text{IPO}} = \mathbb{E}_{(x, y^+, y^-) \sim \mathcal{D}} \left[\left(\Delta_\theta(x, y^+, y^-) - \frac{1}{2\beta} \right)^2 \right].$$

The finite target keeps KL regularization active at all β and removes DPO’s unbounded-margin failure mode.

AOT. AOT (Melnik et al., 2024) replaces per-pair comparison with a distributional one. For each completion, define the reference-corrected reward $r(x, y) = \log \pi_\theta(y | x) - \log \pi_{\text{ref}}(y | x)$. Within a minibatch of B pairs, AOT sorts the chosen and rejected rewards independently and applies a DPO-style logistic to the resulting quantile gaps:

$$\mathcal{L}_{\text{AOT}} = \frac{1}{B} \sum_{i=1}^B -\log \sigma(\beta (r_{(i)}^+ - r_{(i)}^-)),$$

where $r_{(i)}^+$ and $r_{(i)}^-$ are the i -th order statistics of the chosen and rejected reward vectors. The sort step is the closed-form discrete optimal-transport plan under a smooth, convex cost (Melnik et al., 2024), encouraging first-order stochastic dominance of the chosen distribution. The objective tolerates isolated mis-ordered examples so long as stochastic dominance holds, which makes it robust to noisy preference labels.

Analysis. DPO, IPO, and AOT differ along two axes: the shape of the loss applied to the reference-corrected margin, and the granularity at which margins are compared. DPO is *per-pair and unbounded*: it drives $\Delta_\theta \rightarrow \infty$ on every pair, separating chosen from rejected as much as the data allows, which makes it the most aggressive fitter of individual labels and the most affected by mislabeled ones. IPO is *per-pair and bounded*: once a pair’s margin hits the fixed target $1/(2\beta)$, its gradient vanishes regardless of preference strength, removing DPO’s unbounded-margin failure mode at the cost of saturating early. AOT is *distributional*: by comparing chosen and rejected rewards at matched ranks within a minibatch, it averages over individual label errors and is the most robust to noise, but it saturates uniformly once the chosen distribution dominates the rejected one. Our Part 2 method exploits this complementarity: AOT’s distribution-shaping signal is most useful while the policy is still not optimal, and DPO’s per-pair signal is most useful later, once only a few hard pairs remain.

3 ONLINE METHODS

Background. All online methods share two components: a per-prompt advantage and a sampled KL regularizer against the frozen reference.

- **Group-relative advantages:** For each prompt x_i we draw G responses, score them with the reward model to get rewards $r_{i,j}$, and form $A_{i,j} = (r_{i,j} - \mu_i)/(\sigma_i + \varepsilon)$, where μ_i, σ_i are the per-prompt mean and standard deviation. The baseline μ_i reduces variance without a learned value network; variants that omit the σ_i rescaling use $A_{i,j} = r_{i,j} - \mu_i$.
- **Sampled KL regularization:** For a sampled token a at state s , let $\delta(a) = \log \pi_{\text{ref}}(a | s) - \log \pi_\theta(a | s)$ and use the nonnegative proxy $\hat{k}(a) = e^{\delta(a)} - \delta(a) - 1$. Its expectation under $a \sim \pi_\theta$ equals the exact token-level KL, and each sample contributes a nonnegative term that avoids cancellation. We average $\hat{k}$ over completion tokens and scale by κ before adding to the surrogate.

Bradley-Terry Reward Model. We fine-tune the base model with a scalar regression head under the Bradley-Terry preference model, giving the cross-entropy objective

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x,y^+,y^-) \sim \mathcal{D}} [\log \sigma(r_\phi(x,y^+) - r_\phi(x,y^-))].$$

The trained $r_\phi(x,y)$ is then frozen and serves as the environment reward for all online methods below.

GRPO. GRPO (Shao et al., 2024) applies a PPO-style clipped surrogate at the token level. With token importance ratio $\rho_{i,t} = \pi_\theta(y_{i,t} | x_i, y_{i,<t}) / \pi_{\theta_{\text{old}}}(y_{i,t} | x_i, y_{i,<t})$ and per-token surrogate $\ell_{i,t} = \min(\rho_{i,t} A_i, \text{clip}(\rho_{i,t}, 1 - \epsilon, 1 + \epsilon) A_i)$, the objective is

$$\mathcal{L}_{\text{GRPO}} = -\mathbb{E} \left[\frac{1}{|y_i|} \sum_t \ell_{i,t} \right] + \kappa \cdot \mathbb{E}[\hat{k}].$$

The completion-token mean prevents longer positive-advantage responses from receiving disproportionately larger gradients, which would otherwise inflate response length over training.

DrGRPO. DrGRPO (Liu et al., 2025) makes two changes to GRPO. Advantages are mean-centered but not divided by σ_i (dividing amplifies prompts whose sampled rewards happen to cluster), and the per-sequence length normalization is removed:

$$\mathcal{L}_{\text{DrGRPO}} = -\mathbb{E} \left[\sum_t \ell_{i,t} \right] + \kappa \cdot \mathbb{E}[\hat{k}].$$

Responses now contribute proportionally to their token count under the same advantage, removing GRPO’s implicit penalty on longer completions.

GSPO. GSPO (Zheng et al., 2025) clips at the sequence level rather than per token. Define the length-normalized sequence log-ratio

$$s_i = \frac{1}{|y_i|} \sum_t (\log \pi_\theta(y_{i,t} | x_i, y_{i,<t}) - \log \pi_{\theta_{\text{old}}}(y_{i,t} | x_i, y_{i,<t})),$$

i.e., the log of the geometric mean of per-token importance ratios. The clipped surrogate is then applied at the response level:

$$\mathcal{L}_{\text{GSPO}} = -\mathbb{E} [\min(e^{s_i} A_i, \text{clip}(e^{s_i}, 1 - \epsilon, 1 + \epsilon) A_i)] + \kappa \cdot \mathbb{E}[\hat{k}].$$

Length normalization in s_i controls the variance of the sequence importance ratio across response lengths and prevents a few high-likelihood tokens from dominating the sequence-level estimate.

4 PART 2 METHODS

In this section, we introduce our improved offline methods. To enhance performance, we employ Pair-Weighting and a hybrid AOT offline approach, combined with reranking via a reward model.

4.1 PAIR-WEIGHTING DPO

We introduce a pair-weighting to account for the varying quality of preference pairs within the dataset. We believe that not all training samples are equally reliable, and assigning identical importance to noisy preference pairs may degrade optimization performance while diminishing the contribution of high-quality data.

To address this issue, we assign an adaptive weight to each preference pair according to its the quality score. Let $q_i \in [0, 1]$ denote the quality score of the i -th preference pair, where larger values indicate higher-confidence or higher-quality data.

To increase the diversity of the weighting distribution and enhance the distinction between low-quality and high-quality samples, we further introduce a nonlinear transformation controlled by a power coefficient γ : $\tilde{q}_i = q_i^\gamma$. Based on the transformed quality score, the weight assigned to the i -th preference pair is defined as

$$w_i = w_{\min} + (w_{\max} - w_{\min}) \tilde{q}_i,$$

where $w_{\min}$ and $w_{\max}$ denote the minimum and maximum allowable pair weights, respectively.

Given a batch of preference losses $\{\mathcal{L}_i\}_{i=1}^N$ and their corresponding weights $\{w_i\}_{i=1}^N$, the final weighted training objective is computed as

$$\mathcal{L}_{\text{weighted}} = \frac{\sum_{i=1}^N w_i \mathcal{L}_i}{\sum_{i=1}^N w_i}.$$

This weighting strategy enables the optimization process to place greater emphasis on high-quality preference pairs while reducing the negative influence of noisy or unreliable samples. As shown in Fig 4, our Pair-Weights DPO method evaluation accuracy is on par or even higher than DPO method.

4.2 HYBRID AOT

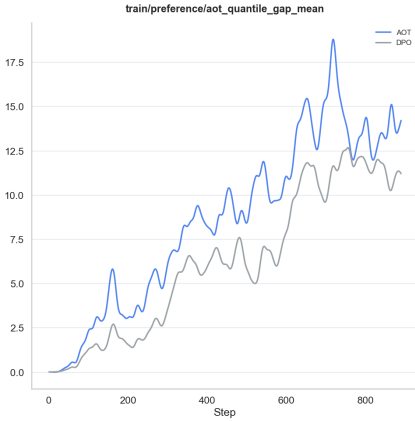


Figure 1: Quantile Gap

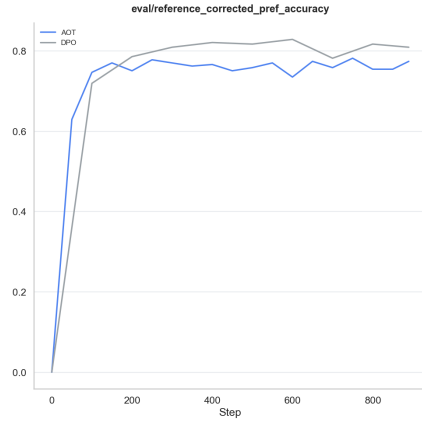


Figure 2: Eval Accuracy

After analyzing the performance of the baselines in Table 1, we observed that DPO and AOT achieved the strongest performance among the three offline baselines.

For DPO, the optimization objective mainly focuses on pairwise ordering. As a result, DPO largely ignores the overall reward distribution. As illustrated in Fig. 1, although DPO achieves strong pairwise preference accuracy, it fails to effectively enlarge the quantile gap, resulting in insufficient separation between high-quality and low-quality samples in the reward space.

In contrast, AOT directly performs distribution-level optimization over quantiles. However, this distributional optimization also discards the pairwise correspondence information between samples. As shown in Fig. 2, the reference-corrected accuracy of AOT is lower than DPO, indicating that AOT may weaken pairwise preference learning.

Therefore, we aim to design an algorithm that combines the advantages of both offline methods. We propose a new hybrid loss function that dynamically balances the AOT loss and the DPO loss using a tunable coefficient α :

$$\mathcal{L} = (1 - \alpha) \mathcal{L}_{\text{AOT}} + \alpha \mathcal{L}_{\text{DPO}}.$$

We hypothesize that dynamically balancing the AOT constraint and the DPO constraint enables the model to simultaneously learn both reward distribution geometry and pairwise preference ordering. As shown in Fig 4, our hybrid method evaluation accuracy is higher than DPO and AOT method.

4.3 RERANKING REWARD MODEL SELECTION

Furthermore, we observe that the trained reward model achieves strong performance in terms of pairwise accuracy. Motivated by this observation, we aim to leverage the reward model to further improve generation quality through response reranking.

We introduce a reranking framework based on the reward model. Assume there are n candidate models, $\mathcal{M} = \{M_1, M_2, \dots, M_n\}$. For a given input prompt x , each candidate model generates k response candidates, $\{y_{i1}, y_{i2}, \dots, y_{ik}\}$, where $i = 1, 2, \dots, n$. As a result, the full candidate set contains a total of $n \times k$ responses. The reward model $R(\cdot)$ then assigns a reward score to each generated response, $s_{ij} = R(x, y_{ij})$. The final output is selected according to the highest reward score:

$$y^* = \arg \max_{y_{ij}} R(x, y_{ij}).$$

Intuitively, this reranking strategy allows the reward model to act as a preference-based selector over multiple candidate responses, thereby improving the overall response quality.

Due to the project constraints, the temperature is fixed to 1 and top- k sampling is disabled. Therefore, in our implementation, we set $k = 1$. Meaning that each candidate model generates only a single response for reranking.

5 EXPERIMENTAL SETUP

Benchmark and base model. We train and evaluate on `wildchat_min4_judged_5k_v1`, a 5000-example instruction-following benchmark with paired LLM-judged responses, split into `train_prefs/test_prefs` (preference pairs) and `train_gen/test_gen` (prompt-only). The base policy and frozen reference π_{ref} are Qwen/Qwen2.5-1.5B-Instruct; all trainable models use LoRA adapters. Full hyperparameters, token budgets, and adapter configuration are in Appendix A.1.

Offline preference training. DPO, IPO, and AOT are trained on `train_prefs` for 3 epochs with AdamW. The preference-loss scale β is 0.005 (DPO), 0.1 (IPO), and 0.2 (AOT), selected on `test_prefs` accuracy.

Reward model. A Bradley-Terry reward model is trained on `train_prefs` with the LM head replaced by a scalar projection. The checkpoint maximizing held-out pair accuracy (step 445) is reused as the reward signal for all online methods and as the Part 2 reranker scorer.

Online RL. GRPO, DrGRPO, and GSPO are trained for 25 optimizer steps on `train_gen` prompts using $G = 4$ rollouts per prompt, scored by the frozen reward model, with a sampled KL penalty against π_{ref} ($\kappa = 0.01$) and PPO-style clipped updates ($\epsilon = 0.2$). All three share identical optimization settings; the only differences are the algorithmic choices in Section 3.

Our methods. Our reranking approach uses four scorers: standard AOT and DPO plus our Hybrid AOT/DPO and Pair-Weight DPO. Hybrid AOT/DPO trains for one epoch with a cosine α schedule from 0 to 1, evaluated at step 200. Pair-Weight DPO weights pairs by dataset confidence scores and linear mapping into range $[0.7, 1.3]$, power 1, also evaluated at step 200.

Evaluation. The primary metric is GPT-5.4 head-to-head win rate against the frozen base model on `public_test_gen_prompts_128.jsonl`, with both policies decoding greedily (temperature 0, `max_new_tokens` = 256); ties and unparseable judgments count as non-wins. The reward model is evaluated by pair accuracy on `public_test_prefs_256.jsonl`.

6 RESULTS

6.1 HEADLINE WIN RATES

Table 1 reports per-seed and aggregate GPT-5.4 win rates across all six baselines plus our Part 2 reranker, together with reward-model pair accuracy on the 256-pair preference set. All methods clear the corresponding autograder thresholds on every seed.

DPO and AOT are statistically indistinguishable on the mean (DPO 0.763, AOT 0.754) but differ sharply in seed-to-seed stability: DPO’s standard deviation of 0.044 is nearly an order of magnitude larger than AOT’s 0.005, making AOT the most reproducible offline method. GRPO is the most seed-sensitive method overall (± 0.061), producing both the strongest single Part 1 number (0.796, seed 0) and a substantially weaker run (0.674, seed 1); we attribute this to the under-converged 25-step budget rather than the loss itself. DrGRPO, GSPO, and the Part 2 reranker are all low-variance (≤ 0.017). The Part 2 reranker reaches 0.865 ± 0.015 , the highest mean in the table, and every individual seed exceeds every Part 1 method’s mean.

Table 1: Win rate against the frozen base model on the 128-prompt evaluation set under the GPT-5.4 judge, reported per seed and as mean $\pm$ one sample standard deviation (with $n - 1$ divisor) across two seeds. The reward-model pair accuracy is computed on the 256-pair preference set; a single reward-model checkpoint is shared across online seeds.

Method	Metric	Seed 0	Seed 1	Seed 2	Mean $\pm$ Std
Bradley-Terry Reward	Pair accuracy	0.844	0.824	0.840	0.836 $\pm$ 0.011
DPO	Win rate	0.735	0.813	0.740	0.763 $\pm$ 0.044
IPO	Win rate	0.639	0.686	0.692	0.672 $\pm$ 0.029
AOT	Win rate	0.759	0.750	0.752	0.754 $\pm$ 0.005
GRPO	Win rate	0.796	0.674	0.727	0.732 $\pm$ 0.061
DrGRPO	Win rate	0.615	0.605	0.611	0.610 $\pm$ 0.005
GSPO	Win rate	0.705	0.725	0.738	0.723 $\pm$ 0.017
Part 2 Offline Reranker	Win rate	0.861	0.852	0.881	0.865 $\pm$ 0.015

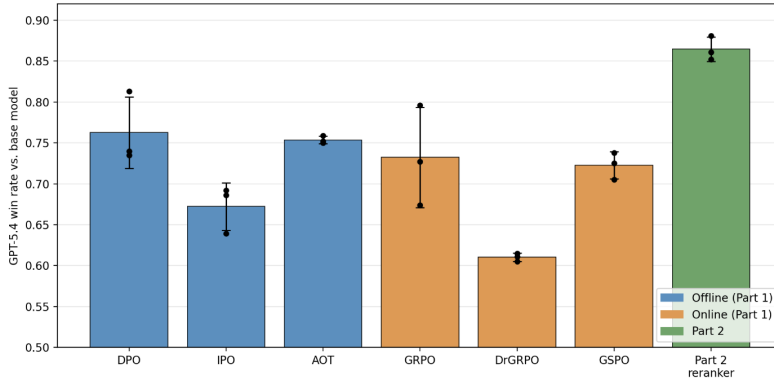


Figure 3: GPT-5.4 head-to-head win rate against the frozen base model, averaged over three random seeds with identical hyperparameters.

6.2 REWARD MODEL

The Bradley-Terry reward model on step 445 achieves a mean pair accuracy of 0.836 ± 0.011 on the held-out `test_prefs` across three training seeds.

We also ablate reward model quality by substituting the mid-training checkpoint (`step_000200`) for the final one (`step_000445`) as the reranking selector, holding the five-policy candidate pool fixed. The earlier checkpoint achieves 0.789 pair accuracy (202/256) on `test_prefs`, 4.7 percentage points below step 445. Under this weaker selector, the five-policy reranker drops from 0.865 to 0.841 win rate, a 5.9% relative reduction in RM accuracy.

6.3 PART 2 RERANKER IMPROVEMENTS

The Part 2 reranker pools four candidates per prompt: two Part 1 baselines (DPO at step 100 and AOT at step 550) and two novel objectives introduced in this work (the AOT-DPO hybrid with a linear schedule and confidence-weighted DPO). To check whether the mean 0.865 is driven by the novel objectives or by an oracle-over-four-baselines effect, we audit the seed-0 reranker output (0.861) and identify the candidate that produced each winning response. The Part 1 baseline candidates share their checkpoints with the corresponding submissions, so under deterministic decoding their responses are byte-identical to `dpo.jsonl` and `aot.jsonl` and can be matched exactly; the remaining two are lumped as “Part 2 novel” in Table 2.

Table 2: Per-candidate origin of each reranked winning response on the 128-prompt evaluation set. The Part 2 reranker pooled four candidates per prompt; the two Part 1 baseline candidates were identified by exact response match against `policy_generations/dpo.jsonl` and `policy_generations/aot.jsonl`.

Candidate origin	Wins	Fraction
Part 2 novel (AOT-DPO hybrid or confidence-weighted DPO)	77	60.2%
DPO step 100 (Part 1 baseline)	31	24.2%
AOT step 550 (Part 1 baseline)	20	15.6%

The novel objectives produce 60.2% of winning candidates, so the reranker is not simply pooling two strong Part 1 checkpoints and benefiting from diversity alone: the novel losses produce responses the reward model prefers more often than either Part 1 baseline. At the same time, Part 1 baselines still account for nearly 40% of winners, indicating that the gain over any single objective is partly a candidate-pool oracle effect rather than a property of the Part 2 losses on their own. Notably, DPO step 100 wins more often than AOT step 550 in the pool despite AOT scoring higher in its single-policy evaluation: the two baselines disagree on different prompts and the reward model exploits that disagreement. Reporting only the reranker win rate would therefore overstate the standalone contribution of the Part 2 objectives. Pair-weighted DPO and the AOT-DPO hybrid each clear the Part 2 threshold as standalone policies (0.79 and 0.78 in our development evaluations), and reranking adds roughly 0.09 over the strongest Part 1 mean (0.865 vs DPO’s 0.763) by selecting among four diverse candidates per prompt.

7 ANALYSIS AND ABLATION STUDIES

We conduct targeted ablations to isolate which components of our Part 2 system drive win-rate gains: two of the most impactful were candidate reranking and the AOT-DPO hybrid schedule. We provide the commands to replicate these ablations in A.2.

7.1 ABLATION 1: MULTI-POLICY RERANKING

Table 3 shows that expanding the candidate pool consistently improves win rate: each addition of a strong policy lifts performance, and the five-policy reranker (0.864) surpasses the best single policy (AOT-hybrid) while drawing on a larger usable set (59 vs. 53.9 prompts).

Table 3: Win rate under the LLM judge for individual policies (top) and multi-policy reranking with a trained reward model (bottom). The candidate pool is grown incrementally in order of decreasing standalone quality: AOT-hybrid, conf-weighted DPO, plain DPO, plain AOT, IPO. *Usable* is the number of evaluation prompts on which both judge passes agreed and produced parseable output.

Method	Win Rate (avg. 3 seeds)	Usable (avg. 3 seeds)
Plain AOT	0.754	57.8
Plain DPO	0.763	48.2
Conf-weighted DPO	0.766	47.4
AOT-hybrid	0.860	53.9
Rerank: 1 policy	0.795	49.3
Rerank: 2 policies	0.801	51.7
Rerank: 3 policies	0.787	56.9
Rerank: 4 policies	0.829	59.1
Rerank: 5 policies	0.864	59.0

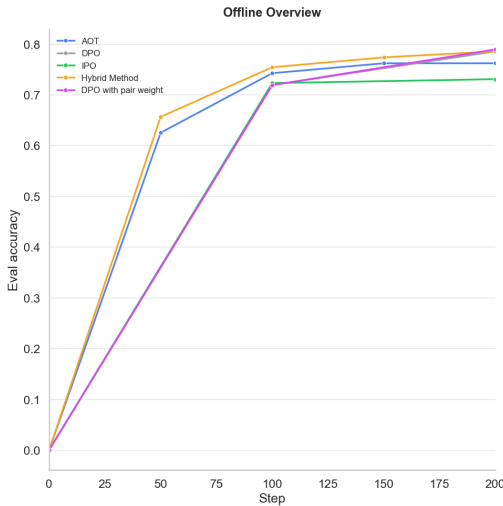


Figure 4: Our Improved Method v.s. Offline Baselines

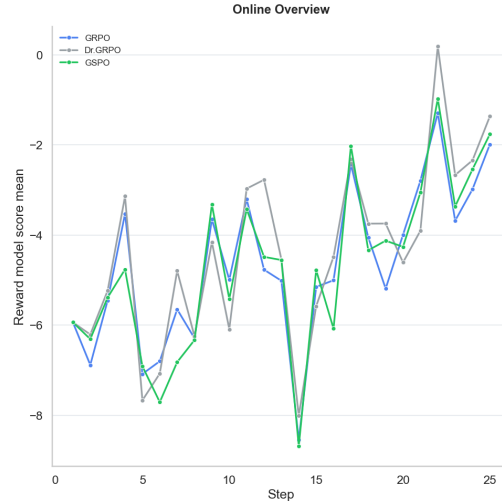


Figure 5: Online Baselines Overview

7.2 ABLATION 2: AOT-DPO HYBRID SCHEDULE

Table 4 isolates the effect of the α trajectory in the AOT-DPO hybrid (Section 2). The cosine forward schedule (A4) reaches 0.861, matching the five-policy reranker as a single-model result.

We identify that schedule shape, not direction, is the key factor. The two linear schedules (A2 forward, A3 reverse) differ by only 0.005, well within judge noise, indicating that the “AOT-early, DPO-late” ordering alone is not sufficient. What distinguishes cosine is its *shape*: at step 100 of 890, cosine gives $\alpha \approx 0.03$ versus $\alpha \approx 0.11$ under linear, so the early phase remains strongly AOT-dominated and the DPO contribution is concentrated into a short terminal burst. A constant midpoint (A1, $\alpha = 0.35$) performs worst at 0.710, consistent with this view: persistent mid-level DPO pressure interferes with early-phase AOT learning without ever delivering the late-phase concentration that drives gains.

Taken together, these results suggest that the hybrid’s benefit is not a weighted average of AOT and DPO gradients, but rather a *temporal separation* between an AOT-dominated initialization phase and a DPO-dominated refinement phase, with the cosine schedule achieving the sharpest version of that separation.

Table 4: Schedule ablation on the AOT-DPO hybrid. All runs train for 3 epochs (≈ 890 steps) with identical hyperparameters except the coefficient α in $\mathcal{L} = (1 - \alpha)\mathcal{L}_{\text{AOT}} + \alpha\mathcal{L}_{\text{DPO}}$. Win rate is evaluated at the final checkpoint against the frozen base model on 128 prompts.

Run	Schedule	α_{start}	α_{end}	Win Rate (avg. 3 seeds)	Usable (avg. 3 seeds)
A1	Constant	0.35	0.35	0.710	51.4
A2	Linear	0.00	1.00	0.762	53.8
A3	Linear	1.00	0.00	0.767	54.1
A4	Cosine	0.00	1.00	0.861	50.2

8 DISCUSSION

8.1 OFFLINE METHODS

Which offline method performed best, and what did the samples reveal? DPO achieved the highest standalone win rate among Part 1 offline methods at 0.763, compared to 0.754 for plain AOT and the IPO baseline. It also produced the most concise outputs: average generated lengths were 192.3, 202.5, and 218.7 tokens for DPO, IPO, and AOT respectively.

Inspecting the generations explains the gap. On factual and structured prompts, DPO responses begin immediately with content. Given the prompt ‘‘Do a deep dive in the following statements about axioms...write the key points down’’, DPO opens with Key Points About Axioms: and proceeds to the numbered list. AOT opens with Sure! Here’s a summary of the key points about axioms in mathematics: before producing an otherwise identical list. On the prompt ‘‘a motivational note for an ailing person’’, DPO returns the note directly: ‘‘Your strength and resilience have already inspired those around you...’’. AOT uses 256 tokens, spending the first sentence announcing Here’s a motivational note for someone who is feeling ill:, then opening with Dear [Name], before the message body, which is then cut off at the token budget. The judge rewards getting to the point; padded preamble costs tokens without adding information.

The failure mode appears on under-specified creative prompts. Given ‘‘Write a Vince McMahon vs Kevin Owens match with Owens’ win in 20 seconds’’, DPO produces: ‘‘Owens delivers an explosive finish to claim victory in just two sentences: ‘Kevin Owens executes his finishing move...’’, where the response describes itself as two sentences rather than writing the match, and then runs the quote into the token limit. AOT produces an actual narrative: ‘‘Kevin Owen executes a flying knee drop...Kevin Owen then pins Vince for the victory. The crowd goes wild.’’ This pattern is consistent with DPO’s pointwise logistic objective: the policy concentrates probability mass on high-reward completions from the training distribution and can produce incoherent text when a prompt falls outside that regime.

AOT’s distributional objective, by contrast, applies a global quantile constraint across the batch, which prevents any single response from collapsing completely while also imposing a verbosity tax in the form of openers and hedging. This complementarity is precisely what motivates the multi-policy reranking approach in Section 4.

IPO provides the clearest case of an internal metric giving a misleading signal. Its squared-loss objective targets a fixed log-ratio margin rather than maximizing the ratio, producing a stable training loss and a reference-corrected preference accuracy on `test_prefs` that climbs to roughly 71% by the final checkpoint. IPO also avoids the token-level pathologies visible in DPO: its average generation length of 202.5 tokens sits between DPO and AOT with no length collapse, and the `train/preference_accuracy` curve is the smoothest of the three offline methods. Yet IPO’s head-to-head win rate is 0.672, the weakest offline result. On the prompt ‘‘Write a Vince McMahon vs Kevin Owens match with Owens’ win in 20 seconds,’’ IPO produces ‘‘Kevin Owens won the match against Vince McMahon in 20 seconds. The brief description is: ‘Kevin Owens defeats Vince

McMahon in a quick match. ” which is grammatical and on-topic but adds no content beyond the prompt itself. The calibration the squared loss imposes prevents mode-seeking collapse at the cost of the sharpness that wins head-to-head, and preference accuracy cannot detect this because it measures only rank order, not whether absolute response quality clears the bar set by the base model.

8.2 ONLINE METHODS

GRPO and GSPO achieved the strongest performance among the three online methods at 0.732 ± 0.061 and 0.723 ± 0.017 , while DrGRPO trailed at 0.610 ± 0.005 .

DrGRPO’s underperformance traces to its two changes from GRPO (Liu et al., 2025): removing σ_i -normalization of advantages and summing the surrogate over tokens rather than averaging. When all completions in a group receive similar reward-model scores, a common regime early in training, the raw advantages are near zero and the effective gradient nearly vanishes; the very low seed variance (± 0.005) is consistent with the optimizer settling into a stable but near-null update regime.

In contrast, GSPO modifies the surrogate construction rather than the advantage estimator. It exponentiates the length-normalized sum of log-ratios to obtain a single geometric-mean ratio per sequence, then applies the PPO clip to that scalar. Sequence-level clipping enforces a single trust-region constraint on the whole completion, which stabilizes optimization at the cost of some granularity. This accounts for both why GSPO is more consistent than GRPO (± 0.017 vs. ± 0.061) and why its mean is slightly lower: the sequence-level constraint prevents the aggressive per-token updates that allowed GRPO to reach 0.796 on its best seed, but also prevents the poor run that sent GRPO to 0.674 on its worst.

8.3 OTHER IDEAS WE TRIED

Beyond our main idea, we also tried other offline idea and we summarized why it failed.

Asymmetric push-pull objectives. We observe that existing offline preference optimization methods, such as DPO and AOT, treat the chosen and rejected responses symmetrically. However, continuously penalizing the rejected response may lead to unnecessary distortion of the reward geometry.

To solve this issue, we attempted to introduce an asymmetric objective that places greater emphasis on improving the chosen response. Concretely, we augment the original preference optimization objective with an additional chosen term:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{AOT/DPO}} + \lambda \mathcal{L}_{\text{chosen}},$$

However, results show that this modification does not improve performance. In particular, we observe that both chosen and rejected rewards decrease simultaneously during training, and the win rate is around 0.69, which is even lower than the original AOT/DPO.

We hypothesize possible reasons for this failure. The WildChat dataset may contain substantial noise. The proposed asymmetric objective in our losses implicitly assumes that the chosen responses are consistently reliable, which may not hold under noisy preference annotations. Second, the loss design itself may be suboptimal: Rather than adding an independent chosen-anchor objective, a nonlinear modulation of the original pairwise loss may be more suitable, since it preserves the relative preference structure while avoiding the assumption that all pairs should contribute equally to optimization.

9 CONCLUSION AND FURTHER WORK

We implemented six contemporary RLHF baselines and a Bradley-Terry reward model on a fixed WildChat benchmark, then introduced three Part 2 improvements: confidence-aware pair-weighting, an AOT-DPO hybrid with a cosine α schedule, and reward-model reranking over a diverse policy pool. The combined pipeline reaches a GPT-5.4 win rate of 0.865 ± 0.015 against the frozen base model, a 0.102 improvement over the strongest Part 1 baseline mean. Ablations show the hybrid’s gain comes from temporal separation between an AOT-dominant early phase and a DPO-dominant late phase rather than from any constant mixture, and the reranker’s gain comes roughly 60% from the novel objectives and 40% from oracle selection over the candidate pool.

Given twice the experimentation budget, three directions are most promising. First, replace the linear-scheduled hybrid in the reranker pool with the cosine-scheduled hybrid. Second, sweep the cosine schedule endpoints beyond $\alpha : 0 \rightarrow 1$ to test whether less aggressive late-stage DPO (e.g., $0.1 \rightarrow 0.9$) preserves the gain with smaller policy drift. Third, improve on online methods like running longer online updates: GRPO’s high seed variance (± 0.061) and the shortness of the 25-step budget show that the online methods are under-converged and would benefit from a 50- or 100-step rerun, possibly combined with replay-based online preference methods (RePO) on top of our best offline policy as a warm start. Reward-model ensembles with pessimistic aggregation and KTO on unpaired desirability labels are further directions we did not have time to explore.

REFERENCES

- Mohammad Gheshlaghi Azar, Zhaohan Daniel Guo, Bilal Piot, Rémi Munos, Mark Rowland, Michal Valko, and Daniele Calandriello. A general theoretical paradigm to understand learning from human preferences. In *International Conference on Artificial Intelligence and Statistics*, pp. 4447–4455. PMLR, 2024.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- Igor Melnyk, Youssef Mroueh, Brian Belgodere, Mattia Rigotti, Apoorva Nitsure, Mikhail Yurochkin, Kristjan Greenewald, Jiri Navratil, and Jarret Ross. Distributional preference alignment of LLMs via optimal transport. In *Advances in Neural Information Processing Systems*, volume 37, pp. 104412–104442, 2024.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pp. 27730–27744, 2022.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.

A APPENDIX

A.1 ADDITIONAL EXPERIMENTAL SETUP DETAILS

Token budgets. Prompts are truncated at 700 tokens. Offline and reward-model training cap responses at 512 tokens. Online rollouts and all submitted generations cap new tokens at 256.

LoRA configuration. All trainable models (offline policies, reward model, online policies) use LoRA with rank $r = 32$, $\alpha = 64$, dropout 0.05, applied to `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj`. Bias parameters are not adapted; gradient checkpointing is enabled. The frozen reference π_{ref} is obtained by disabling the LoRA adapter, avoiding a second copy of the base model.

Offline preference training. AdamW with $(\beta_1, \beta_2) = (0.9, 0.95)$, weight decay 0, gradient clipping at global norm 1.0, learning rate 5×10^{-5} , linear warmup over the first 100 steps. Per-device batch size 4 pairs, gradient accumulation 4 (effective batch 16 pairs), 3 epochs.

Reward model. Same LoRA configuration as the offline policies; the LM head is replaced by a linear projection from the final-token hidden state to a scalar score. AdamW, learning rate 3×10^{-5} , per-device batch size 8, gradient accumulation 4 (effective batch 32), 3 epochs. Checkpoints saved periodically; best held-out pair accuracy at step 445.

Online RL. At each iteration we sample 16 prompts and draw $G = 4$ rollouts per prompt with temperature 0.8, `top_p = 0.95`, `top_k` disabled, `min_new_tokens = 32`, `max_new_tokens = 256`. The σ_i rescaling is enabled for GRPO and GSPO and disabled for DrGRPO. PPO updates: clip $\epsilon = 0.2$, two epochs per rollout batch, minibatch size 8, learning rate 1×10^{-5} , gradient clipping at global norm 0.5, gradient accumulation 2, advantage clip ± 5 .

Diagnostics. In addition to the primary win-rate metric, we track held-out preference accuracy on `test_prefs`, reward-model pair accuracy and margin, approximate KL of the policy against the reference, on-policy reward-model scores during online training, and periodic qualitative inspection of samples.

Compute. Training runs on Modal cloud GPUs (NVIDIA H100s and H200s). A single full 128-prompt GPT-5.4 evaluation costs approximately \$1.07 via the OpenAI API. The Gradescope autograder (five-submission cap) was treated as a confirmation step; local GPT-5.4 evaluations and Weights & Biases were used for the main debugging loop.

A.2 PART 2 ABLATION COMMANDS

We provide the exact commands used to replicate the ablation results.

A.2.1 ABLATION 1 COMMANDS

Plain DPO

```
uv run modal run scripts/modal_train.py::train_remote \
  -- --algo dpo \
  --model_name Qwen/Qwen2.5-1.5B-Instruct \
  --dataset_name /vol/synthetic_datasets/wildchat_min4_judged_5k_v1 \
  --train_split train_prefs --eval_split test_prefs \
  --generation_split test_gen \
  --output_dir /vol/runs/wildchat_min4_judged_5k_dpo_beta01_v1 \
  --beta 0.1 \
  --per_device_train_batch_size 4 --grad_accum_steps 4 --lr 5e-5 \
  --num_train_epochs 3 --max_prompt_tokens 700 --max_response_tokens 512 \
  --generation_eval_limit 32 --generation_eval_max_new_tokens 256 \
  --generation_eval_every 100 --eval_interval 100 --save_interval 100 \
  --wandb_enabled --wandb_project llm-rl-final-project \
  --wandb_name wildchat_min4_judged_5k_dpo_beta01_v1
```

Plain AOT

```
uv run modal run scripts/modal_train.py::train_remote \
  -- --algo aot \
  --model_name Qwen/Qwen2.5-1.5B-Instruct \
  --dataset_name /vol/synthetic_datasets/wildchat_min4_judged_5k_v1 \
  --train_split train_prefs --eval_split test_prefs \
  --generation_split test_gen \
  --output_dir /vol/runs/wildchat_min4_judged_5k_aot_beta02_v1 \
  --beta 0.2 \
  --per_device_train_batch_size 4 --grad_accum_steps 4 --lr 5e-5 \
  --num_train_epochs 3 --max_prompt_tokens 700 --max_response_tokens 512 \
  --generation_eval_limit 32 --generation_eval_max_new_tokens 256 \
  --generation_eval_every 50 --eval_interval 50 --save_interval 50 \
  --wandb_enabled --wandb_project llm-rl-final-project \
  --wandb_name wildchat_min4_judged_5k_aot_beta02_v1
```

IPO

```
uv run modal run scripts/modal_train.py::train_remote \
  -- --algo ipo \
  --model_name Qwen/Qwen2.5-1.5B-Instruct \
  --dataset_name /vol/synthetic_datasets/wildchat_min4_judged_5k_v1 \
  --train_split train_prefs --eval_split test_prefs \
  --generation_split test_gen \
```

```

--output_dir /vol/runs/wildchat_min4_judged_5k_ipo_v1 \
--beta 0.1 \
--per_device_train_batch_size 4 --grad_accum_steps 4 --lr 5e-5 \
--num_train_epochs 3 --max_prompt_tokens 700 --max_response_tokens 512 \
--generation_eval_limit 32 --generation_eval_max_new_tokens 256 \
--generation_eval_every 100 --eval_interval 100 --save_interval 100 \
--wandb_enabled --wandb_project llm-rl-final-project \
--wandb_name wildchat_min4_judged_5k_ipo_v1

```

Confidence-Weighted DPO

```

uv run modal run scripts/modal_train.py::train_remote \
-- --algo dpo \
--model_name Qwen/Qwen2.5-1.5B-Instruct \
--dataset_name /vol/synthetic_datasets/wildchat_min4_judged_5k_v1 \
--train_split train_prefs --eval_split test_prefs \
--generation_split test_gen \
--output_dir /vol/runs/wildchat_min4_judged_5k_dpo_beta01_v1_conf_weight \
--beta 0.1 \
--pair_weight_mode confidence --pair_weight_power 1.0 \
--pair_min_weight 0.7 --pair_max_weight 1.3 \
--per_device_train_batch_size 4 --grad_accum_steps 4 --lr 5e-5 \
--num_train_epochs 3 --max_prompt_tokens 700 --max_response_tokens 512 \
--generation_eval_limit 32 --generation_eval_max_new_tokens 256 \
--generation_eval_every 100 --eval_interval 100 --save_interval 100 \
--wandb_enabled --wandb_project llm-rl-final-project \
--wandb_name wildchat_min4_judged_5k_dpo_beta01_v1_conf_weight

```

AOT-Hybrid

```

uv run modal run scripts/modal_train.py::train_remote \
-- --algo aot-hybrid \
--model_name Qwen/Qwen2.5-1.5B-Instruct \
--dataset_name /vol/synthetic_datasets/wildchat_min4_judged_5k_v1 \
--train_split train_prefs --eval_split test_prefs \
--generation_split test_gen \
--output_dir /vol/runs/wildchat_min4_judged_5k_a_hyb_sched_a4_cosine \
--beta 0.2 --aot_hybrid_schedule cosine \
--aot_hybrid_dpo_weight_start 0.0 --aot_hybrid_dpo_weight_final 1.0 \
--pair_weight_mode confidence --pair_weight_power 1.0 \
--pair_min_weight 0.7 --pair_max_weight 1.3 \
--per_device_train_batch_size 4 --grad_accum_steps 4 --lr 5e-5 \
--num_train_epochs 3 --max_prompt_tokens 700 --max_response_tokens 512 \
--generation_eval_limit 32 --generation_eval_max_new_tokens 256 \
--generation_eval_every 50 --eval_interval 50 --save_interval 25 \
--wandb_enabled --wandb_project llm-rl-final-project \
--wandb_name aot_hybrid_sched_a4_cosine_swing

```

The candidate pool is grown in order of decreasing standalone quality. All five commands use the same reward model checkpoint (step_000445).

Pool size 1: AOT-Hybrid only

```

uv run modal run scripts/modal_train.py::build_policy_reranked_submission_remote \
-- --model_name Qwen/Qwen2.5-1.5B-Instruct \
--adapter_path \
/vol/runs/wildchat_min4_judged_5k_a_hyb_sched_a4_cosine/checkpoints/step_000890/adapter \
--reward_model_name Qwen/Qwen2.5-1.5B-Instruct \
--reward_adapter_path \
/vol/runs/wildchat_min4_judged_5k_reward_model_v1/checkpoints/step_000445/adapter \
--prompts_jsonl /root/project/public_eval/public_test_gen_prompts_128.jsonl \
--output_jsonl /vol/submissions/groupD_rerank_1.jsonl \
--max_prompt_tokens 700 --max_new_tokens 256 \
--max_response_tokens 512 --temperature 0 --top_p 1

```

Pool size 2: + Conf-Weighted DPO

```

uv run modal run scripts/modal_train.py::build_policy_reranked_submission_remote \
-- --model_name Qwen/Qwen2.5-1.5B-Instruct \
--adapter_path \
/vol/runs/wildchat_min4_judged_5k_a_hyb_sched_a4_cosine/checkpoints/step_000890/adapter, \
/vol/runs/wildchat_min4_judged_5k_dpo_beta01_v1_conf_weight/checkpoints/step_000200/adapter \
--reward_model_name Qwen/Qwen2.5-1.5B-Instruct \
--reward_adapter_path \
/vol/runs/wildchat_min4_judged_5k_reward_model_v1/checkpoints/step_000445/adapter \
--prompts_jsonl /root/project/public_eval/public_test_gen_prompts_128.jsonl \
--output_jsonl /vol/submissions/groupD_rerank_2.jsonl \
--max_prompt_tokens 700 --max_new_tokens 256 \
--max_response_tokens 512 --temperature 0 --top_p 1

```

Pool size 3: + Plain AOT

```
uv run modal run scripts/modal_train.py::build_policy_reranked_submission_remote \
-- --model_name Qwen/Qwen2.5-1.5B-Instruct \
--adapter_path \
/vol/runs/wildchat_min4_judged_5k_a_hyb_sched_a4_cosine/checkpoints/step_000890/adapter,\
/vol/runs/wildchat_min4_judged_5k_dpo_beta01_v1_conf_weight/checkpoints/step_000200/adapter,\
/vol/runs/wildchat_min4_judged_5k_aot_beta02_v1/checkpoints/step_000550/adapter \
--reward_model_name Qwen/Qwen2.5-1.5B-Instruct \
--reward_adapter_path \
/vol/runs/wildchat_min4_judged_5k_reward_model_v1/checkpoints/step_000445/adapter \
--prompts_jsonl /root/project/public_eval/public_test_gen_prompts_128.jsonl \
--output_jsonl /vol/submissions/groupD_rerank_3.jsonl \
--max_prompt_tokens 700 --max_new_tokens 256 \
--max_response_tokens 512 --temperature 0 --top_p 1
```

Pool size 4: + Plain DPO

```
uv run modal run scripts/modal_train.py::build_policy_reranked_submission_remote \
-- --model_name Qwen/Qwen2.5-1.5B-Instruct \
--adapter_path \
/vol/runs/wildchat_min4_judged_5k_a_hyb_sched_a4_cosine/checkpoints/step_000890/adapter,\
/vol/runs/wildchat_min4_judged_5k_dpo_beta01_v1_conf_weight/checkpoints/step_000200/adapter,\
/vol/runs/wildchat_min4_judged_5k_aot_beta02_v1/checkpoints/step_000550/adapter,\
/vol/runs/wildchat_min4_judged_5k_dpo_beta01_v1/checkpoints/step_000100/adapter \
--reward_model_name Qwen/Qwen2.5-1.5B-Instruct \
--reward_adapter_path \
/vol/runs/wildchat_min4_judged_5k_reward_model_v1/checkpoints/step_000445/adapter \
--prompts_jsonl /root/project/public_eval/public_test_gen_prompts_128.jsonl \
--output_jsonl /vol/submissions/groupD_rerank_4.jsonl \
--max_prompt_tokens 700 --max_new_tokens 256 \
--max_response_tokens 512 --temperature 0 --top_p 1
```

Pool size 5: + IPO

```
uv run modal run scripts/modal_train.py::build_policy_reranked_submission_remote \
-- --model_name Qwen/Qwen2.5-1.5B-Instruct \
--adapter_path \
/vol/runs/wildchat_min4_judged_5k_a_hyb_sched_a4_cosine/checkpoints/step_000890/adapter,\
/vol/runs/wildchat_min4_judged_5k_dpo_beta01_v1_conf_weight/checkpoints/step_000200/adapter,\
/vol/runs/wildchat_min4_judged_5k_aot_beta02_v1/checkpoints/step_000550/adapter,\
/vol/runs/wildchat_min4_judged_5k_dpo_beta01_v1/checkpoints/step_000100/adapter,\
/vol/runs/wildchat_min4_judged_5k_ipo_v1/checkpoints/step_000300/adapter \
--reward_model_name Qwen/Qwen2.5-1.5B-Instruct \
--reward_adapter_path \
/vol/runs/wildchat_min4_judged_5k_reward_model_v1/checkpoints/step_000445/adapter \
--prompts_jsonl /root/project/public_eval/public_test_gen_prompts_128.jsonl \
--output_jsonl /vol/submissions/groupD_rerank_5.jsonl \
--max_prompt_tokens 700 --max_new_tokens 256 \
--max_response_tokens 512 --temperature 0 --top_p 1
```

A.2.2 ABLATION 2 COMMANDS

A1: Midpoint Baseline (Constant)

```
uv run modal run scripts/modal_train.py::train_remote \
-- --algo aot-hybrid \
--model_name Qwen/Qwen2.5-1.5B-Instruct \
--dataset_name /vol/synthetic_datasets/wildchat_min4_judged_5k_v1 \
--train_split train_prefs --eval_split test_prefs \
--generation_split test_gen \
--output_dir /vol/runs/wildchat_min4_judged_5k_a_hyb_sched_a1_const_mid \
--beta 0.2 --aot_hybrid_schedule constant \
--aot_hybrid_dpo_weight_start 0.35 --aot_hybrid_dpo_weight_final 0.35 \
--pair_weight_mode confidence --pair_weight_power 1.0 \
--pair_min_weight 0.7 --pair_max_weight 1.3 \
--per_device_train_batch_size 4 --grad_accum_steps 4 --lr 5e-5 \
--num_train_epochs 3 --max_prompt_tokens 700 --max_response_tokens 512 \
--generation_eval_limit 32 --generation_eval_max_new_tokens 256 \
--generation_eval_every 50 --eval_interval 50 --save_interval 25 \
--wandb_enabled --wandb_project llm-rl-final-project \
--wandb_name aot_hybrid_sched_a1_constant_mid
```

A2: Linear Swing

```
uv run modal run scripts/modal_train.py::train_remote \
-- --algo aot-hybrid \
--model_name Qwen/Qwen2.5-1.5B-Instruct \
```

```

--dataset_name /vol/synthetic_datasets/wildchat_min4_judged_5k_v1 \
--train_split train_prefs --eval_split test_prefs \
--generation_split test_gen \
--output_dir /vol/runs/wildchat_min4_judged_5k_a_hyb_sched_a2_lin \
--beta 0.2 --aot_hybrid_schedule linear \
--aot_hybrid_dpo_weight_start 0.0 --aot_hybrid_dpo_weight_final 1.0 \
--pair_weight_mode confidence --pair_weight_power 1.0 \
--pair_min_weight 0.7 --pair_max_weight 1.3 \
--per_device_train_batch_size 4 --grad_accum_steps 4 --lr 5e-5 \
--num_train_epochs 3 --max_prompt_tokens 700 --max_response_tokens 512 \
--generation_eval_limit 32 --generation_eval_max_new_tokens 256 \
--generation_eval_every 50 --eval_interval 50 --save_interval 25 \
--wandb_enabled --wandb_project llm-rl-final-project \
--wandb_name aot_hybrid_sched_a2_linear_swing

```

A3: Reverse Swing

```

uv run modal run scripts/modal_train.py::train_remote \
-- --algo aot-hybrid \
--model_name Qwen/Qwen2.5-1.5B-Instruct \
--dataset_name /vol/synthetic_datasets/wildchat_min4_judged_5k_v1 \
--train_split train_prefs --eval_split test_prefs \
--generation_split test_gen \
--output_dir /vol/runs/wildchat_min4_judged_5k_a_hyb_sched_a3_lin_rev \
--beta 0.2 --aot_hybrid_schedule linear \
--aot_hybrid_dpo_weight_start 1.0 --aot_hybrid_dpo_weight_final 0.0 \
--pair_weight_mode confidence --pair_weight_power 1.0 \
--pair_min_weight 0.7 --pair_max_weight 1.3 \
--per_device_train_batch_size 4 --grad_accum_steps 4 --lr 5e-5 \
--num_train_epochs 3 --max_prompt_tokens 700 --max_response_tokens 512 \
--generation_eval_limit 32 --generation_eval_max_new_tokens 256 \
--generation_eval_every 50 --eval_interval 50 --save_interval 25 \
--wandb_enabled --wandb_project llm-rl-final-project \
--wandb_name aot_hybrid_sched_a3_linear_reverse

```

A4: Cosine Swing

```

uv run modal run scripts/modal_train.py::train_remote \
-- --algo aot-hybrid \
--model_name Qwen/Qwen2.5-1.5B-Instruct \
--dataset_name /vol/synthetic_datasets/wildchat_min4_judged_5k_v1 \
--train_split train_prefs --eval_split test_prefs \
--generation_split test_gen \
--output_dir /vol/runs/wildchat_min4_judged_5k_a_hyb_sched_a4_cosine \
--beta 0.2 --aot_hybrid_schedule cosine \
--aot_hybrid_dpo_weight_start 0.0 --aot_hybrid_dpo_weight_final 1.0 \
--pair_weight_mode confidence --pair_weight_power 1.0 \
--pair_min_weight 0.7 --pair_max_weight 1.3 \
--per_device_train_batch_size 4 --grad_accum_steps 4 --lr 5e-5 \
--num_train_epochs 3 --max_prompt_tokens 700 --max_response_tokens 512 \
--generation_eval_limit 32 --generation_eval_max_new_tokens 256 \
--generation_eval_every 50 --eval_interval 50 --save_interval 25 \
--wandb_enabled --wandb_project llm-rl-final-project \
--wandb_name aot_hybrid_sched_a4_cosine_swing

```